UNIT-III No SQL Databases

NoSQL DATABASES No SQL databases: Mongo DB: Introduction – Features – Data types – Mongo DB Query language – CRUD operations – Arrays – Functions: Count – Sort – Limit – Skip – Aggregate – Map Reduce. Cursors – Indexes – Mongo Import – Mongo Export

NoSQL Database

A **NoSQL database** is a database that does not primarily store data in traditional relational tables like SQL databases. The name is commonly understood as **"Not Only SQL."**

Main types

- **Document database** — stores data as JSON-like documents.
  Examples: MongoDB, CouchDB
- **Key-value database** — stores data as key → value pairs.
  Examples: Redis, DynamoDB
- **Wide-column database** — stores large amounts of distributed data using column families.
  Examples: Cassandra, HBase
- **Graph database** — stores entities as nodes and their relationships as edges.
  Examples: Neo4j, Amazon Neptune

Simple example

A document database might store a customer like this:

```
{
  "id": 101,
  "name": "Ravi",
  "age": 25,
  "city": "Chennai",
  "skills": ["Java", "Python"]
}
```

Unlike a traditional SQL table, the structure can be more flexible.

Tables and rows	Documents, key-value, graphs, etc.
Usually fixed schema	Often flexible schema
SQL query language	Database-specific queries/APIs
Strong relational model	Different models depending on database
Excellent for structured relational data	Excellent for flexible/distributed data
Examples: MySQL, PostgreSQL	Examples: MongoDB, Redis, Cassandra

In simple terms: SQL databases organize data mainly into tables, while NoSQL databases use other structures that can be better suited to flexible, very large, or distributed applications.

MongoDB: Introduction

MongoDB is a popular **NoSQL, document-oriented database**. Instead of storing data in tables and rows like MySQL or PostgreSQL, MongoDB stores data as **documents**, which are similar to JSON objects.
MongoDB develops MongoDB.

1. What is MongoDB?

MongoDB is a database management system designed to store and manage **semi-structured data**.
A MongoDB document looks like:

```
{

  "name": "Ravi",

  "age": 25,

  "city": "Chennai",

  "skills": ["Java", "Python", "MongoDB"]

}
```

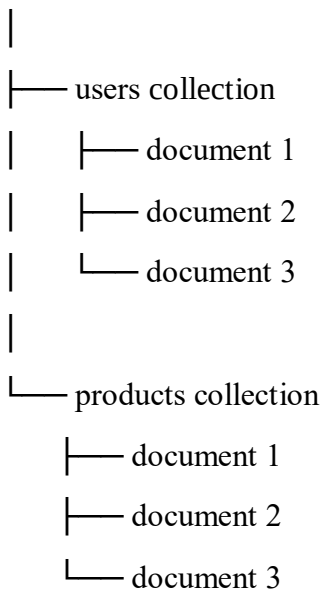
ID	Name	Age	City
1	Ravi	25	Chennai

2. MongoDB terminology

The following terms are important:

MongoDB	**Relational Database**
Database	Database
Collection	Table
Document	Row
Field	Column
_id	Primary key

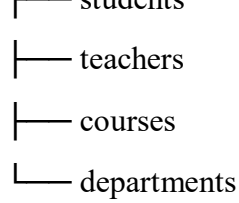
MongoDB Database



3. Database

A **database** is a container for collections.
For example:
CollegeDB

CollegeDB



4. Collection

A **collection** is similar to a table in SQL.
For example:
Students

could contain several student documents:

```
{
  "name": "Ravi",
  "age": 20,
  "course": "Computer Science"
}
```

and:

```
{
  "name": "Priya",
  "age": 21,
  "course": "Information Technology"
}
```

Unlike a traditional SQL table, documents in the same collection don't necessarily have exactly the same fields.

5. Document

A **document** is the basic unit of data in MongoDB.

Example:

```
{
  "_id": 101,
  "name": "Ravi",
  "age": 20,
  "email": "ravi@example.com",
  "address": {
    "city": "Chennai",
    "state": "Tamil Nadu"
  }
}
```

Documents can contain:

- Strings
- Numbers
- Boolean values
- Arrays
- Objects
- Dates
- Null values
- Other MongoDB-supported data types

This makes documents suitable for storing complex, nested information.

6.field

Every MongoDB document normally has a unique _id field.

Example:

```
{
  "_id": 101,
  "name": "Ravi"
}
```

If you don't provide an _id, MongoDB automatically generates one, commonly as an **ObjectId**.

Example:

```
{
  "_id": ObjectId("64f...")
}
```

The _id uniquely identifies the document within its collection.

7. BSON

MongoDB stores documents internally using **BSON**.

BSON stands for **Binary JSON**.

Although MongoDB documents are commonly represented using JSON-like syntax, BSON provides additional data types and is optimized for storage and processing.

For example, BSON supports types such as:

- String
- Integer
- Double
- Boolean

- Date
- Array
- Embedded document
- ObjectId
- Binary data

8. Basic MongoDB operations

MongoDB supports the basic **CRUD** operations:

C — Create

```
db.students.insertOne({
  name: "Ravi",
  age: 20
})
```

R — Read

```
db.students.find()
```

U — Update

```
db.students.updateOne(
  { name: "Ravi" },
  { $set: { age: 21 } }
)
```

D — Delete

```
db.students.deleteOne({
  name: "Ravi"
})
```

So:

CRUD

```
├── Create
├── Read
├── Update
└── Delete
```

9. Important features of MongoDB

Flexible schema

Documents don't have to follow exactly the same structure.

Document-oriented

Related information can be stored together in a document.

Scalability

MongoDB supports **horizontal scaling** through mechanisms such as sharding.

Replication

MongoDB supports replication to improve availability and redundancy.

Indexing

Indexes can improve query performance.

Aggregation

MongoDB provides an aggregation framework for processing and analyzing data.

Transactions

MongoDB supports transactions, including multi-document transactions, when transactional behavior is required.

---

10. Advantages of MongoDB

- Flexible document structure
- Easy to represent nested data
- Good support for large-scale applications
- Horizontal scaling
- Replication and high availability
- Powerful querying and aggregation
- Suitable for rapidly changing application requirements

12. Limitations

MongoDB isn't automatically the best choice for every application.
Potential challenges include:

- Complex relational data may require careful document design.
- Data duplication can occur when information is embedded.
- Poorly designed indexes can hurt performance.
- Developers need to understand MongoDB's document model and consistency/transaction behavior.

MongoDB Data Types

MongoDB supports several **BSON data types**. BSON means **Binary JSON**, and it extends JSON with additional data types such as dates, ObjectIds, and binary data.

1. String

Used to store text.

```
{
  name: "Ravi",
  city: "Chennai"
}
```

Here, "Ravi" and "Chennai" are strings.

2. Integer

Used to store whole numbers.

```
{
  age: 25,
  quantity: 10
}
```

MongoDB supports different integer representations depending on the value and driver.

3. Double

Used to store floating-point numbers.

```
{
  price: 99.99,
  temperature: 36.5
}
```

4. Boolean

Stores either true or false.

```
{
  isStudent: true,
  isActive: false
}
```

5. Array

Stores multiple values in a single field.

```
{
  name: "Ravi",
  skills: ["Java", "Python", "MongoDB"]
}
```

The skills field is an array.

Arrays can also contain objects:

```
{
  name: "Ravi",
  marks: [85, 90, 78]
}
```

6. Embedded Document

A document can contain another document.

```
{
  name: "Ravi",
  address: {
    city: "Chennai",
    state: "Tamil Nadu"
  }
}
```

Here, address is an embedded document.

This is one of the important features of MongoDB because related data can be stored together.

7. ObjectId

ObjectId is commonly used for MongoDB's _id field.

Example:

```
{
  _id: ObjectId("507f1f77bcf86cd799439011"),
  name: "Ravi"
}
```

An ObjectId is designed to provide a unique identifier for a document.

If you don't specify _id when inserting a document, MongoDB generally generates an ObjectId automatically.

8. Date

Used to store dates and times.

```
{
  name: "Ravi",
  joiningDate: ISODate("2026-08-24T10:00:00Z")
}
```

Dates are useful for:

- Registration dates
- Birth dates
- Login times
- Order dates
- Timestamps

9. Null

Represents the absence of a value.

```
{
  name: "Ravi",
  phone: null
}
```

Here, the phone field currently has no value.

10. Binary Data

MongoDB supports **binary data**, which can be used for data such as binary files or application-specific binary content.

```
{
  fileData: <binary data>
}
```

For large files, MongoDB also provides **GridFS**, which is designed for storing and retrieving files larger than the normal BSON document-size limit.

11. Regular Expression

MongoDB supports regular expressions for pattern matching.

For example:

```
{
  name: /^Ravi/
}
```

This can be used to find values that match a particular text pattern.

12. Decimal128

Decimal128 is designed for **precise decimal arithmetic**.

It is particularly useful for applications such as:

- Financial calculations
- Currency
- Accounting
- Precise measurements

For example:

```
{
  price: Decimal128("199.99")
}
```

This can be preferable to ordinary floating-point representation when exact decimal precision matters.

MongoDB Query Language

MongoDB uses a query language based on **JavaScript-style syntax** to work with documents. It is commonly used through the MongoDB Shell (mongosh) and MongoDB drivers.

Unlike SQL, MongoDB does not use commands such as SELECT, INSERT INTO, and UPDATE. Instead, it uses methods such as find(), insertOne(), and updateOne().

1. Basic structure

MongoDB data is organized as:

Database
↓
Collection
↓
Documents

↓
Fields
For example, suppose we have a students collection:

```
{
  name: "Ravi",
  age: 20,
  department: "Computer Science",
  marks: 85
}
```

2. Selecting a database

Use use to switch to a database:

```
use college
```

You can see the current database with:

```
db
```

3. Insert operations

Insert one document

```
db.students.insertOne({
  name: "Ravi",
  age: 20,
  department: "Computer Science"
})
```

Insert multiple documents

```
db.students.insertMany([
  {
    name: "Ravi",
    age: 20,
    department: "Computer Science"
  },
  {
    name: "Priya",
    age: 21,
    department: "Information Technology"
  }
])
```

4. Read operations

The main method for retrieving documents is find().

Find all documents

```
db.students.find()
```

Find one document

```
db.students.findOne({
  name: "Ravi"
})
```

Find documents with a condition

```
db.students.find({
  age: 20
})
```

This returns students whose age is 20.

5. Comparison operators

MongoDB provides comparison operators.

Operator	Meaning
$eq	Equal
$ne	Not equal
$gt	Greater than
$gte	Greater than or equal
$lt	Less than
$lte	Less than or equal
$in	Matches any value in an array
$nin	Does not match values in an array

Greater than

```
db.students.find({
  age: { $gt: 20 }
})
```

Greater than or equal

```
db.students.find({
  marks: { $gte: 80 }
})
```

Less than

```
db.students.find({
  marks: { $lt: 50 }
})
```

Not equal

```
db.students.find({
  department: { $ne: "Computer Science" }
})
```

Match multiple possible values

```
db.students.find({
  department: {
    $in: ["Computer Science", "Information Technology"]
  }
})
```

6. Logical operators

MongoDB provides logical operators such as:

- $and
- $or
- $not
- $nor

AND

Find students who are older than 18 **and** have marks greater than 80:

```
db.students.find({
  $and: [
    { age: { $gt: 18 } },
    { marks: { $gt: 80 } }
  ]
})
```

For simple conditions, MongoDB can often express AND without explicitly using $and:

```
db.students.find({
  age: { $gt: 18 },
  marks: { $gt: 80 }
})
```

OR

```
db.students.find({
  $or: [
    { department: "Computer Science" },
    { department: "Mathematics" }
  ]
})
```

7. Projection

Projection controls which fields MongoDB returns.

Suppose the document is:

```
{
  name: "Ravi",
  age: 20,
  department: "Computer Science",
  marks: 85
}
```

To return only name and marks:

```
db.students.find(
  {},
  {
    name: 1,
    marks: 1
  }
)
```

The result contains the selected fields, along with _id by default.

To exclude _id:

```
db.students.find(
  {},
  {
    _id: 0,
    name: 1,
    marks: 1
  }
)
```

8. Update operations

updateOne()
Update one matching document:

```
db.students.updateOne(
  { name: "Ravi" },
  { $set: { marks: 90 } }
)
```

updateMany()
Update multiple documents:

```
db.students.updateMany(
  { department: "Computer Science" },
  { $set: { university: "ABC University" } }
)
```

9. Delete operations

Delete one document

```
db.students.deleteOne({
  name: "Ravi"
})
```

Delete multiple documents

```
db.students.deleteMany({
  age: { $lt: 18 }
})
```

Be careful with deleteMany() because it can remove many documents.

10. Sorting

Use sort() to order results.
Ascending:

```
db.students.find().sort({
  marks: 1
})
```

Descending:

```
db.students.find().sort({
  marks: -1
})
```

Here:
1 → Ascending
-1 → Descending

11. Limit

limit() restricts the number of results.

```
db.students.find().limit(5)
```

This returns at most five documents.

13. Skip

skip() skips a specified number of documents.

```
db.students.find().skip(5).limit(5)
```

This can be useful for basic pagination.

14. Counting documents

You can count matching documents:

```
db.students.countDocuments({
  marks: { $gte: 80 }
})
```

15. Querying arrays

Suppose we have:

```
{
  name: "Ravi",
  skills: ["Java", "Python", "MongoDB"]
}
```

Find documents containing "Python":

```
db.students.find({
  skills: "Python"
})
```

You can also use $all when multiple values must be present:

```
db.students.find({
  skills: {
    $all: ["Java", "Python"]
  }
})
```

16. Querying embedded documents

Suppose:

```
{
  name: "Ravi",
  address: {
    city: "Chennai",
    state: "Tamil Nadu"
  }
}
```

You can query the nested city field using **dot notation**:

```
db.students.find({
  "address.city": "Chennai"
})
```

17. Regular expressions

MongoDB supports regular-expression queries.

For example:

```
db.students.find({
  name: /^R/
})
```

This finds names beginning with R.

18. Aggregation

The **aggregation framework** is used for more complex data processing.

For example, calculate the average marks:

```
db.students.aggregate([
  {
    $group: {
      _id: null,
      averageMarks: {
        $avg: "$marks"
      }
    }
  }
])
```

Common aggregation stages include:

- $match — filter documents
- $group — group documents
- $sort — sort results
- $project — select/reshape fields
- $limit — restrict results
- $lookup — combine data from another collection

Example:

```
db.students.aggregate([
  {
    $match: {
      marks: { $gte: 80 }
    }
  },
  {
    $sort: {
      marks: -1
    }
  }
])
```

This first selects students with marks of at least 80 and then sorts them by marks in descending order.

CRUD Operations in MongoDB

CRUD stands for:

- **C — Create**
- **R — Read**
- **U — Update**
- **D — Delete**

These are the four basic operations used to manage data in MongoDB.
Assume we have a database called college and a collection called students.

```
use college
```

1. Create

Create means adding new documents to a collection.
insertOne()
Inserts one document:

```
db.students.insertOne({
  name: "Ravi",
  age: 20,
  department: "Computer Science",
  marks: 85
})
```

MongoDB automatically creates an _id if you don't provide one.

Example document:

```
{
```

```
  "_id": ObjectId("..."),
  "name": "Ravi",
  "age": 20,
  "department": "Computer Science",
  "marks": 85
}
```

InsertMany()

Inserts multiple documents:

```
db.students.insertMany([
  {
    name: "Ravi",
    age: 20,
    department: "Computer Science",
    marks: 85
  },
  {
    name: "Priya",
    age: 21,
    department: "Information Technology",
    marks: 90
  },
  {
    name: "Arun",
    age: 19,
    department: "Mathematics",
    marks: 78
  }
])
```

2. Read

Read means retrieving documents from a collection.

find()

To retrieve all students:

```
db.students.find()
```

To find students from Computer Science:

```
db.students.find({
  department: "Computer Science"
})
```

findOne()

To retrieve one matching document:

```
db.students.findOne({
  name: "Ravi"
})
```

Using comparison operators

Find students with marks greater than 80:

```
db.students.find({
  marks: { $gt: 80 }
})
```

Find students with marks greater than or equal to 80:

```
db.students.find({
  marks: { $gte: 80 }
})
```

Find students younger than 21:

```
db.students.find({
  age: { $lt: 21 }
})
```

3. Projection

Projection allows you to specify which fields should be returned.

For example:

```
db.students.find(
  { department: "Computer Science" },
  {
    _id: 0,
    name: 1,
    marks: 1
  }
)
```

Result:

```
{
  "name": "Ravi",
  "marks": 85
}
```

Here:

- 1 means include the field.
- 0 means exclude the field.

4. Update

Update means modifying existing documents.

updateOne()

Suppose Ravi's marks need to be changed from 85 to 95:

```
db.students.updateOne(
  { name: "Ravi" },
  {
    $set: {
      marks: 95
    }
  }
)
```

$inc

To increase Ravi's marks by 5:

```
db.students.updateOne(
  { name: "Ravi" },
  {
    $inc: {
      marks: 5
    }
  }
)
```

If his marks were 85, they become 90.

updateMany()

Update multiple documents.

For example, add a college field to all Computer Science students:

```
db.students.updateMany(
  { department: "Computer Science" },
  {
    $set: {
      college: "ABC College"
    }
  }
)
```

Every matching document is updated.

5. Delete

Delete means removing documents.

deleteOne()

Delete one student:

```
db.students.deleteOne({
  name: "Ravi"
})
```

Only one matching document is removed.

deleteMany()

Delete all students whose marks are below 40:

```
db.students.deleteMany({
  marks: { $lt: 40 }
})
```

All matching documents are removed.

Be careful with deleteMany(), especially when the filter is empty:

```
db.students.deleteMany({})
```

This can delete **all documents in the collection**.

Arrays in MongoDB

An **array** in MongoDB is a data type that allows you to store **multiple values inside a single field**.

For example, a student can have multiple skills:

```
{
  name: "Ravi",
  age: 20,
  skills: ["Java", "Python", "MongoDB"]
}
```

Here, skills is an array containing three values.

1. Creating an Array

You can create an array when inserting a document:

```
db.students.insertOne({
  name: "Ravi",
  skills: ["Java", "Python", "MongoDB"]
})
```

An array can contain different types of values:

```
{
  name: "Ravi",
  marks: [85, 90, 78]
}
```

It can also contain documents:

```
{
  name: "Ravi",
  courses: [
    {
      name: "MongoDB",
      duration: 3
    },
    {
      name: "Python",
      duration: 4
    }
  ]
}
```

2. Reading Arrays

Suppose we have:

```
{
  name: "Ravi",
  skills: ["Java", "Python", "MongoDB"]
}
```

To find students who have Python:

```
db.students.find({
  skills: "Python"
})
```

MongoDB can match an array element directly.

3. $in Operator

$in finds documents where an array contains **at least one** of the specified values.

```
db.students.find({
  skills: {
    $in: ["Python", "JavaScript"]
  }
})
```

This finds students who have either Python or JavaScript.

4. $all Operator

$all requires the array to contain **all specified values**.

```
db.students.find({
  skills: {
    $all: ["Java", "Python"]
  }
})
```

This finds students who have both Java and Python.

5. Adding an Element with $push

The $push operator adds a new value to an array.

Suppose:

```
{
  name: "Ravi",
  skills: ["Java", "Python"]
}
```

Add MongoDB:

```
db.students.updateOne(
  { name: "Ravi" },
  {
    $push: {
      skills: "MongoDB"
    }
  }
)
```

The result becomes:

```
{
  name: "Ravi",
  skills: ["Java", "Python", "MongoDB"]
}
```

6. Adding Multiple Elements

You can use $each with $push:

```
db.students.updateOne(
  { name: "Ravi" },
  {
    $push: {
      skills: {
        $each: ["JavaScript", "React"]
      }
    }
  }
)
```

7. Adding an Element Only if It Doesn't Exist

Use $addToSet.

```
db.students.updateOne(
  { name: "Ravi" },
  {
    $addToSet: {
      skills: "Python"
    }
  }
)
```

If Python is already present, MongoDB doesn't add another copy.

This is useful when you want an array to behave more like a set of unique values.

8. Removing an Element with $pull

$pull removes matching values from an array.

```
db.students.updateOne(
  { name: "Ravi" },
```

```
  {
    $pull: {
      skills: "Python"
    }
  }
)
```
Before:
["Java", "Python", "MongoDB"]
After:
["Java", "MongoDB"]

9. Removing the First or Last Element

The $pop operator removes an element.
Remove the **last** element:

```
db.students.updateOne(
  { name: "Ravi" },
  {
    $pop: {
      skills: 1
    }
  }
)
```

Remove the **first** element:

```
db.students.updateOne(
  { name: "Ravi" },
  {
    $pop: {
      skills: -1
    }
  }
)
```

10. Array of Documents

Arrays can contain complete documents.
Example:

```
{
  name: "Ravi",
  courses: [
    {
      course: "MongoDB",
      marks: 90
    },
    {
      course: "Python",
      marks: 85
    }
  ]
}
```

You can query nested array documents.
For example:

```
db.students.find({
  "courses.course": "MongoDB"
```

})

11. Array Indexing

MongoDB arrays use **zero-based indexing**.
For:

```
{
  skills: ["Java", "Python", "MongoDB"]
}
```

the indexes are:
0 → Java
1 → Python
2 → MongoDB
You can access/query an array element using dot notation:

```
db.students.find({
  "skills.0": "Java"
})
```

This looks at the first element of the skills array.

12. Array Size

The $size operator matches arrays with a specific number of elements.
For example:

```
db.students.find({
  skills: {
    $size: 3
  }
})
```

This finds students whose skills array contains exactly three elements.

Functions in MongoDB

In MongoDB, **functions** can mean reusable JavaScript functions in the MongoDB shell or **database operations/methods** that perform tasks on documents.
For basic MongoDB study, it is useful to understand both.

1. Functions for CRUD operations

MongoDB provides methods that work like functions.

Create

```
db.students.insertOne({
  name: "Ravi",
  age: 20
})
db.students.insertMany([
  { name: "Ravi", age: 20 },
  { name: "Priya", age: 21 }
])
```

Read

```
db.students.find()
db.students.findOne({
  name: "Ravi"
})
```

Update

```
db.students.updateOne(
  { name: "Ravi" },
  { $set: { age: 21 } }
)
```

Delete

```
db.students.deleteOne({
  name: "Ravi"
})
```

These methods form the basic **CRUD functions/methods** of MongoDB.

2. Common MongoDB methods

Some frequently used methods are:

Method	Purpose
insertOne()	Insert one document
insertMany()	Insert multiple documents
find()	Find documents
findOne()	Find one document
updateOne()	Update one document
updateMany()	Update multiple documents
deleteOne()	Delete one document
deleteMany()	Delete multiple documents
countDocuments()	Count matching documents
sort()	Sort results
limit()	Limit results
skip()	Skip results

3. Counting documents

countDocuments() counts documents matching a condition.

```
db.students.countDocuments()
```

Count students with marks greater than 80:

```
db.students.countDocuments({
  marks: { $gt: 80 }
})
```

4. Sorting

The sort() method sorts query results.

Ascending:

```
db.students.find().sort({
```

```
  marks: 1
})
```
Descending:
```
db.students.find().sort({
  marks: -1
})
```
5. Limiting results
limit() restricts the number of documents returned.
```
db.students.find().limit(5)
```
This returns at most five documents.
6. Skipping documents
skip() skips a specified number of documents.
```
db.students.find().skip(5)
```
You can combine it with limit():
```
db.students.find()
  .skip(5)
  .limit(5)
```
This is commonly used for pagination.
7. Aggregation functions
MongoDB's aggregation framework provides operators for calculations and data processing.
Some important operators are:

- $sum — calculates a total
- $avg — calculates an average
- $min — finds the minimum
- $max — finds the maximum
- $count — counts documents
- $group — groups documents

For example, calculate the average marks:
```
db.students.aggregate([
  {
    $group: {
      _id: null,
      averageMarks: {
        $avg: "$marks"
      }
    }
  }
])
```
Calculate total marks:
```
db.students.aggregate([
  {
    $group: {
      _id: null,
      totalMarks: {
        $sum: "$marks"
      }
    }
```

```
  }
])
```

8. String-related functions/operators

MongoDB provides aggregation expressions for manipulating strings.
For example, $toUpper converts text to uppercase:

```
db.students.aggregate([
  {
    $project: {
      name: { $toUpper: "$name" }
    }
  }
])
```

If the name is:
Ravi
the result can be:
RAVI
Similarly, $toLower converts text to lowercase.

9. Array-related functions/operators

MongoDB has many aggregation expressions for arrays.
For example, $size returns the number of elements:

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      skillCount: { $size: "$skills" }
    }
  }
])
```

If:
skills: ["Java", "Python", "MongoDB"]
then:
skillCount: 3

10. User-defined JavaScript functions

In appropriate MongoDB shell/application contexts, JavaScript functions can also be written.
For example:

```
function add(a, b) {
  return a + b;
}

add(10, 20);
```

Result:
30
However, **application-side functions are generally preferred for business logic** rather than treating MongoDB itself as a general-purpose JavaScript execution environment.